```
// === App Shell (js/app.js) ===

/**Ð
 * vÊfz\ n8b T –Æ - App ShellÐ
 * @author ch2077Ð
 * @version 3.2.0Ð
 * @date 2025-05-10 etT 21k>n8b Ð
 *Ð
 * n8b hFg¶ÿ j!WWS g¶g„ÿ kĬN*n8b rìzËjsj!WWÐ
 * ~ßN u T}Thg ÿ init(container, statsCb) !’ destroy()Ð
 * ~ß‹¡VÞŒ ÿ updateStats({key:value}) R7e°˜v•èr¶` h Ð
 */Ð
const app = {Ð
  currentGame: null,Ð
  games: [Ð
    {Ð
      id: 'memory',Ð
      name: '‹°_Æ•ûrL',Ð
      icon: 'Ø>Ýà',Ð
      desc: '•û_ SarGb~R0vøT v„’M[ùÿ € šŒO`v„‹°_ÆR›ÿ ',Ð
      tags: ['‹°_Æ', '’M[ù'],Ð
      module: MemoryGameÐ
    },Ð
    {Ð
      id: 'snake',Ð
      name: '•*T †Ç',Ð
      icon: 'Ø=Ü ',Ð
      desc: '~ĬQx•*T †Çn8b ÿ T c‰˜ßri•Š•••ŠY'ÿ ',Ð
      tags: ['~ĬQx', 'ˆWg:'],Ð
      module: SnakeGameÐ
    },Ð
    {Ð
      id: '2048',Ð
      name: '2048',Ð
      icon: 'Ø=Ý"',Ð
      desc: 'nÑR¨T ^vep[We¹WWÿ c b •¾R0 2048ÿ ',Ð
      tags: ['ep[W', '{Vue'],Ð
      module: Game2048Ð
    },Ð
    {Ð
      id: 'minesweeper',Ð
      name: 'bk–÷',Ð
      icon: 'Ø=Ü£',Ð
      desc: '~ĬQxbk–÷n8b ÿ ••_ W0–÷b~Qúb@g [‰Qhh<ÿ ',Ð
      tags: ['~ĬQx', 'c¨t '],Ð
      module: MinesweeperGameÐ
    },Ð
    {Ð
      id: 'puzzle',Ð
      name: 'nÑWWbüVþ',Ð
      icon: 'Ø>Ýé',Ð
      desc: 'yûR¨nÑWW•ØSŸVþrG˜z^•ÿ ~ĬQxv„ 15 büVþÿ ',Ð
      tags: ['•;•’', 'c’^•'],Ð
      module: PuzzleGameÐ
```

```
  },Ð
  {Ð
    id: 'tetris',Ð
    name: 'OÄ•We¯e¹WW',Ð
    icon: 'Ø>Ýñ',Ð
    desc: '~ÏQxOÄ•We¯e¹WWÿ mˆ–detˆL_—R SG~§ÿ ',Ð
    tags: ['~ÏQx', 'ˆWg:'],Ð
    module: TetrisGameÐ
  },Ð
  {Ð
    id: 'tictactoe',Ð
    name: 'N•[WhË',Ð
    icon: '+U',Ð
    desc: '~ÏQxN [PhËÿ N  AI [ùb w Œ QH•Þb N ~¿ÿ ',Ð
    tags: ['{Vue', '[ùb '],Ð
    module: TicTacToeGameÐ
  },Ð
  {Ð
    id: 'numberguess',Ð
    name: 'ep[Ws Œ ',Ð
    icon: 'Ø<ß¯',Ð
    desc: 's Qú 1~100 NK•ôv„y^yØep[Wÿ ƒ VôO • n •)\ ÿ ',Ð
    tags: ['•;•’', 'c¨t '],Ð
    module: NumberGuessGameÐ
  },Ð
  {Ð
    id: 'breakout',Ð
    name: 'bSx WW',Ð
    icon: 'Ø<ßÓ',Ð
    desc: '~ÏQxbSx WWÿ yûR¨c!g•SÍ_9\ t mˆ–db@g x WWÿ ',Ð
    tags: ['~ÏQx', 'ˆWg:'],Ð
    module: BreakoutGameÐ
  },Ð
  {Ð
    id: 'reaction',Ð
    name: 'SÍ^”mK‹Õ',Ð
    icon: '&¡',Ð
    desc: 'mK‹ÕO`v„SÍ^”• ^¦ÿ w R0~ÿ,rzËSsp¹Qûÿ ',Ð
    tags: ['SÍ^”', 'c b '],Ð
    module: ReactionGameÐ
  },Ð
  {Ð
    id: 'sudoku',Ð
    name: 'eprì',Ð
    icon: 'Ø=Ý"',Ð
    desc: '~ÏQxN][«h<ep[Wc¨t ÿ Xknáb@g zzh<ÿ ',Ð
    tags: ['ep[W', 'c¨t '],Ð
    module: SudokuGameÐ
  },Ð
  {Ð
    id: 'flappy',Ð
    name: '\ ž ˜Þ˜Þ',Ð
    icon: 'Ø=Ü$',Ð
    desc: 'p¹Qû‹©\ ž ˜Þ•wÿ z••Ç{¡•Sÿ •Š•Ü•ŠY}ÿ ',Ð
    tags: ['O •ò', 'ˆWg:'],Ð
```

```
    module: FlappyGameÐ
  },Ð
  {Ð
    id: 'fruitcatch',Ð
    name: 'c¥l4gœ',Ð
    icon: 'Ø<ßN',Ð
    desc: 'nÑR¨{î[Pc¥OOl4gœÿ \ _Ã••_ p¸_9ÿ ',Ð
    tags: ['O •ò', 'SÍ^”'],Ð
    module: FruitCatchGameÐ
  },Ð
  {Ð
    id: 'colormatch',Ð
    name: '˜œ,rS9‘M',Ð
    icon: 'Ø<ß¨',Ð
    desc: '_ë• R$e-e‡[W˜œ,rf/T&N T yðN •ôÿ 30yÒc b ÿ ',Ð
    tags: ['SÍ^”', '• R›'],Ð
    module: ColorMatchGameÐ
  },Ð
  {Ð
    id: 'delta',Ð
    name: 'N ‰Òm2‘MˆÅ',Ð
    icon: 'Ø<ß–þ ',Ð
    desc: 'N ‰Òm2ˆLR¨–•g:‘MˆÅVhÿ W0Vþ0 ^rTX0 gªh°Qh–•g:ÿ QÆY QúQûÿ ',Ð
    tags: ['–•g:', '‘MˆÅ'],Ð
    module: DeltaGameÐ
  },Ð
  {Ð
    id: 'bubble',Ð
    name: 'lálá\ bK',Ð
    icon: 'Ø>þç',Ð
    desc: '~ÏQxláláŸ™ÿ w„QÆ\ Qûÿ mˆ–d3N*T ,rlláÿ ',Ð
    tags: ['~ÏQx', 'Flash'],Ð
    module: BubbleGameÐ
  },Ð
  {Ð
    id: 'invaders',Ð
    name: 'Y*zzOµue€ ',Ð
    icon: 'Ø=Ü~',Ð
    desc: '~ÏQxˆWg:\ Qûÿ mˆpmY f ,0– ÿ OÝSkW0t ÿ ',Ð
    tags: ['ˆWg:', '\ Qû'],Ð
    module: InvadersGameÐ
  },Ð
  {Ð
    id: 'wordle',Ð
    name: 's SU‹Í',Ð
    icon: 'Ø=Ý$',Ð
    desc: '6k!g:O s Qú5[WkÍ,ñe‡SU‹Íÿ [WkÍSØ,rcÐy:ÿ ',Ð
    tags: ['SU‹Í', 'c¨t '],Ð
    module: WordleGameÐ
  },Ð
  {Ð
    id: 'simon',Ð
    name: '‹°_Æpo',Ð
    icon: 'Ø=Ü¡',Ð
    desc: '‹°OOpoQI˜z^•^v‘ÍY ÿ •Šge•Š••c b g•–Pÿ ',Ð
```

```
      tags: ['‹°_Æ', 'c b '],Ð
      module: SimonGameÐ
    },Ð
    {Ð
      id: 'whack',Ð
      name: 'bSW0Ÿ ',Ð
      icon: 'Ø=Ü9',Ð
      desc: '30yÒ–PeöbSW0Ÿ ÿ w<u¾bK_ë•ŠY •ŠY}ÿ ',Ð
      tags: ['SÍ^”', 'O •ò'],Ð
      module: WhackGameÐ
    },Ð
    {Ð
      id: 'match3',Ð
      name: '[•wómˆmˆNP',Ð
      icon: 'Ø=ÜŽ',Ð
      desc: 'N¤cbvø•»[•wóÿ 3N*NåN •Þ~¿mˆ–dÿ •Þ• •ÞQûÿ ',Ð
      tags: ['mˆ–d', '{Vue'],Ð
      module: Match3GameÐ
    }Ð
  ],Ð
Ð
  init() {Ð
    this.renderGameGrid();Ð
    this.bindEvents();Ð
  },Ð
Ð
  renderGameGrid() {Ð
    const grid = document.getElementById('gameGrid');Ð
    grid.innerHTML = this.games.map(g => `Ð
      <div class="game-card" data-game="${g.id}">Ð
        <div class="game-icon">${g.icon}</div>Ð
        <div class="game-name">${g.name}</div>Ð
        <div class="game-desc">${g.desc}</div>Ð
        <div class="game-tags">${g.tags.map(t => `<span class="tag">${t}</span>`).
join('')}</div>Ð
      </div>Ð
    `).join('');Ð
  },Ð
Ð
  bindEvents() {Ð
    document.getElementById('gameGrid').addEventListener('click', (e) => {Ð
      const card = e.target.closest('.game-card');Ð
      if (card) {Ð
        const gameId = card.dataset.game;Ð
        this.openGame(gameId);Ð
      }Ð
    });Ð
Ð
    document.getElementById('backBtn').addEventListener('click', () => {Ð
      this.closeGame();Ð
    });Ð
  },Ð
Ð
  openGame(gameId) {Ð
    const gameDef = this.games.find(g => g.id === gameId);Ð
```

```
    if (!gameDef) return;Ð
Ð
    this.currentGame = gameDef;Ð
Ð
    document.getElementById('homePage').style.display = 'none';Ð
    document.getElementById('gamePage').style.display = 'block';Ð
    document.getElementById('backBtn').style.display = 'block';Ð
    document.getElementById('gameTitle').textContent = gameDef.name;Ð
Ð
    const controls = document.getElementById('gameControls');Ð
    controls.innerHTML = `Ð
      <div class="game-stats" id="gameStats"></div>Ð
      <button class="btn small" id="gameRestart">‘Íe°_ YË</button>Ð
    `;Ð
    document.getElementById('gameRestart').addEventListener('click', () => {Ð
      this.startGame();Ð
    });Ð
Ð
    document.getElementById('gameArea').innerHTML = '';Ð
    this.startGame();Ð
  },Ð
Ð
  closeGame() {Ð
    if (this.currentGame && this.currentGame.module && this.currentGame.module.des
troy) {Ð
      this.currentGame.module.destroy();Ð
    }Ð
    this.currentGame = null;Ð
    document.getElementById('homePage').style.display = 'block';Ð
    document.getElementById('gamePage').style.display = 'none';Ð
    document.getElementById('backBtn').style.display = 'none';Ð
    document.getElementById('gameArea').innerHTML = '';Ð
  },Ð
Ð
  startGame() {Ð
    if (!this.currentGame) return;Ð
    const area = document.getElementById('gameArea');Ð
    area.innerHTML = '';Ð
    this.currentGame.module.init(area, this.updateStats.bind(this));Ð
  },Ð
Ð
  updateStats(stats) {Ð
    const el = document.getElementById('gameStats');Ð
    if (el) {Ð
      el.innerHTML = Object.entries(stats).map(([k, v]) =>Ð
        `<div class="stat">${k}: <span>${v}</span></div>`Ð
      ).join('');Ð
    }Ð
  }Ð
};Ð
Ð
document.addEventListener('DOMContentLoaded', () => app.init());Ð


// === HTML (index.html) ===
```

```
<!DOCTYPE html>Ð
<html lang="zh-CN">Ð
<head>Ð
<meta charset="UTF-8">Ð
<meta name="viewport" content="width=device-width, initial-scale=1.0">Ð
<title>vÊfz\ n8b </title>Ð
<link rel="stylesheet" href="css/style.css">Ð
</head>Ð
<body>Ð
<div id="app">Ð
  <header class="app-header">Ð
    <div class="header-left">Ð
      <span class="logo">Ø<ß®</span>Ð
      <h1>vÊfz\ n8b </h1>Ð
    </div>Ð
    <button class="btn-back" id="backBtn" style="display:none">!• •ÔVÞ</button>Ð
  </header>Ð
Ð
  <main id="mainContent">Ð
    <!-- Home page - game grid -->Ð
    <div id="homePage" class="home-page">Ð
      <div class="hero">Ð
        <h2>• béN N*n8b _ YËT'ÿ </h2>Ð
        <p>O •òe>g~ÿ •;p¼` ~ô</p>Ð
      </div>Ð
      <div class="game-grid" id="gameGrid">Ð
        <!-- Populated by JS -->Ð
      </div>Ð
    </div>Ð
Ð
    <!-- Game container -->Ð
    <div id="gamePage" class="game-page" style="display:none">Ð
      <div class="game-header" id="gameHeader">Ð
        <h2 id="gameTitle"></h2>Ð
        <div class="game-controls" id="gameControls"></div>Ð
      </div>Ð
      <div class="game-area" id="gameArea">Ð
        <!-- Each game renders here -->Ð
      </div>Ð
    </div>Ð
  </main>Ð
Ð
  <footer class="app-footer">Ð
    <p>vÊfz\ n8b T –Æ - –•eöges©ÿ </p>Ð
  </footer>Ð
</div>Ð
Ð
<script src="js/games/memory.js"></script>Ð
<script src="js/games/snake.js"></script>Ð
<script src="js/games/2048.js"></script>Ð
<script src="js/games/minesweeper.js"></script>Ð
<script src="js/games/puzzle.js"></script>Ð
<script src="js/games/tetris.js"></script>Ð
<script src="js/games/tictactoe.js"></script>Ð
```

```
<script src="js/games/numberguess.js"></script>Ð
<script src="js/games/breakout.js"></script>Ð
<script src="js/games/reaction.js"></script>Ð
<script src="js/games/sudoku.js"></script>Ð
<script src="js/games/flappy.js"></script>Ð
<script src="js/games/fruitcatch.js"></script>Ð
<script src="js/games/colormatch.js"></script>Ð
<script src="js/games/delta.js"></script>Ð
<script src="js/games/bubble.js"></script>Ð
<script src="js/games/invaders.js"></script>Ð
<script src="js/games/wordle.js"></script>Ð
<script src="js/games/simon.js"></script>Ð
<script src="js/games/whack.js"></script>Ð
<script src="js/games/match3.js"></script>Ð
<script src="js/app.js"></script>Ð
</body>Ð
</html>Ð


// === Breakout (js/games/breakout.js) ===

/**
 * Breakout (bSx WW) - 5 Qs•ïQsj!_
 * @author ch2077
 * @version 2.2.0
 * @date 2025-04-18 R Qe5Qs•ïQsÿ kÏQsx WWc’^ N T
 * @date 2025-05-03 Œ N†c!g•_9t POyû{—lÕÿ hitPosf \ dxfô•êq6
 *
 * ~ÏQxbSx WWn8b ÿ c!g•SÍ_9\ t mˆ–db@g x WWSs•ÇQs
 * SÍ_9‰Ò^¦u1t QûN-c!g•v„OM•nQ³[šÿ -0.5~0.5f \ R0-4~4ÿ
 * kÏQsx WWˆLep0 R ep0 t • • Xžÿ {,5Qs7ˆL10R +•…šØ•
 * u T}3gaÿ c‰t bcT}ÿ T}\=game over
 */
const BreakoutGame = {
  canvas: null, ctx: null,
  width: 480, height: 320,
  paddle: { x: 200, w: 80, h: 12 },  // c!g•[½80šØ12
  ball: { x: 240, y: 280, r: 6, dx: 3, dy: -3 },  // t SJ_„6ÿ R YËeœN
  bricks: [],
  score: 0,
  lives: 3,
  level: 1,
  maxLevel: 5,
  running: false,
  loop: null,
  container: null,
  statsCb: null,

  levelConfigs: [
    { rows: 4, cols: 7, speed: 3, colors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a0'
] },
    { rows: 5, cols: 8, speed: 3.5, colors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a
0','#54a0ff'] },
    { rows: 5, cols: 9, speed: 4, colors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a0'
,'#54a0ff'] },
```

```
    { rows: 6, cols: 9, speed: 4.5, colors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a
0','#54a0ff','#a29bfe'] },
    { rows: 7, cols: 10, speed: 5, colors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a0
','#54a0ff','#a29bfe','#ee5a24'] },
  ],

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0;
    this.lives = 3;
    this.level = 1;
    this.running = true;

    const w = Math.min(480, container.clientWidth - 48);
    const h = w * 0.67;
    this.width = w;
    this.height = h;

    this.canvas = document.createElement('canvas');
    this.canvas.className = 'game-canvas';
    this.canvas.width = w;
    this.canvas.height = h;
    this.ctx = this.canvas.getContext('2d');

    this.paddle = { x: w / 2 - 40, w: 80, h: 12 };

    container.innerHTML = '';
    container.appendChild(this.canvas);
    const inst = document.createElement('div');
    inst.className = 'instructions';
    inst.textContent = '!• !’ yûR¨c!g• | Ÿ h  / ‰æ\ObÖR¨';
    container.appendChild(inst);

    this.showLevelIntro(() => {
      this.buildLevel();
      this.updateStats();
      document.addEventListener('keydown', this.handleKey);
      document.addEventListener('mousemove', this.handleMouse);
      this.canvas.addEventListener('touchmove', this.handleTouch, {passive:false})
;
      this.canvas.addEventListener('touchstart', this.handleTouch, {passive:false}
);
      this.loop = setInterval(() => this.tick(), 16);
    });
  },

  buildLevel() {
    const cfg = this.levelConfigs[this.level - 1];
    const cols = cfg.cols, rows = cfg.rows;
    const bw = (this.width - 16) / cols, bh = 16;
    this.bricks = [];
    for (let r = 0; r < rows; r++)
      for (let c = 0; c < cols; c++)
        this.bricks.push({
```

```
          x: c * bw + 8, y: r * (bh + 4) + 30,
          w: bw - 4, h: bh, alive: true,
          color: cfg.colors[r % cfg.colors.length]
        });
    this.ball = {
      x: this.width / 2, y: this.height - 40, r: 6,
      dx: (Math.random() - 0.5) * cfg.speed, dy: -cfg.speed
    };
  },

  showLevelIntro(cb) {
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">bSx WW</div>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    setTimeout(() => { if (div.parentNode) div.remove(); cb(); }, 1500);
  },

  showLevelClear(cb) {
    this.running = false;
    clearInterval(this.loop);
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">'  •ÇQsÿ </div><div class="level-title">QÆY
•ÛQe{, ${this.level + 1} Qs</div>`;
    this.container.appendChild(div);
    setTimeout(() => { if (div.parentNode) div.remove(); cb(); }, 1500);
  },

  handleKey(e) {
    if (e.key === 'ArrowLeft') BreakoutGame.paddle.x -= 20;
    if (e.key === 'ArrowRight') BreakoutGame.paddle.x += 20;
    BreakoutGame.paddle.x = Math.max(0, Math.min(BreakoutGame.width - BreakoutGame
.paddle.w, BreakoutGame.paddle.x));
  },

  handleMouse(e) {
    const rect = BreakoutGame.canvas.getBoundingClientRect();
    BreakoutGame.paddle.x = e.clientX - rect.left - BreakoutGame.paddle.w / 2;
    BreakoutGame.paddle.x = Math.max(0, Math.min(BreakoutGame.width - BreakoutGame
.paddle.w, BreakoutGame.paddle.x));
  },

  handleTouch(e) {
    e.preventDefault();
    const rect = BreakoutGame.canvas.getBoundingClientRect();
    BreakoutGame.paddle.x = e.touches[0].clientX - rect.left - BreakoutGame.paddle
.w / 2;
    BreakoutGame.paddle.x = Math.max(0, Math.min(BreakoutGame.width - BreakoutGame
.paddle.w, BreakoutGame.paddle.x));
  },

  tick() {
```

```
    if (!this.running) return;
    const b = this.ball, p = this.paddle, spd = this.levelConfigs[this.level - 1].
speed;

    b.x += b.dx;
    b.y += b.dy;

    if (b.x - b.r <= 0 || b.x + b.r >= this.width) b.dx *= -1;
    if (b.y - b.r <= 0) b.dy *= -1;

    if (b.y + b.r >= this.height - p.h && b.y + b.r <= this.height - p.h + 10 &&
        b.x > p.x && b.x < p.x + p.w) {
      b.dy = -Math.abs(b.dy);
      const hitPos = (b.x - p.x) / p.w;
      b.dx = (hitPos - 0.5) * spd * 2;
      if (Math.abs(b.dy) < spd) b.dy = -spd;
    }

    if (b.y - b.r > this.height) {
      this.lives--;
      this.updateStats();
      if (this.lives <= 0) { this.gameOver(); return; }
      b.x = this.width / 2; b.y = this.height - 40;
      b.dx = (Math.random() - 0.5) * spd; b.dy = -spd;
    }

    for (const brick of this.bricks) {
      if (!brick.alive) continue;
      if (b.x + b.r > brick.x && b.x - b.r < brick.x + brick.w &&
          b.y + b.r > brick.y && b.y - b.r < brick.y + brick.h) {
        brick.alive = false;
        b.dy *= -1;
        this.score += 10;
        this.updateStats();
        if (this.bricks.every(br => !br.alive)) {
          if (this.level >= this.maxLevel) { this.gameOver(true); }
          else { this.levelClear(); }
          return;
        }
        break;
      }
    }

    this.draw();
  },

  levelClear() {
    this.running = false;
    clearInterval(this.loop);
    document.removeEventListener('keydown', this.handleKey);
    document.removeEventListener('mousemove', this.handleMouse);
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">'  {, ${this.level} Qs• •Çÿ </div><div class
="level-title">Q£Y •ÛQe{, ${this.level + 1} Qs</div>`;
```

```
    this.container.appendChild(div);
    setTimeout(() => {
      if (div.parentNode) div.remove();
      this.level++;
      this.running = true;
      document.addEventListener('keydown', this.handleKey);
      document.addEventListener('mousemove', this.handleMouse);
      this.canvas.addEventListener('touchmove', this.handleTouch, {passive:false})
;
      this.canvas.addEventListener('touchstart', this.handleTouch, {passive:false}
);
      this.showLevelIntro(() => {
        this.buildLevel();
        this.updateStats();
        this.loop = setInterval(() => this.tick(), 16);
      });
    }, 1800);
  },

  draw() {
    const c = this.ctx;
    c.clearRect(0, 0, this.width, this.height);
    for (const b of this.bricks) {
      if (!b.alive) continue;
      c.fillStyle = b.color;
      c.fillRect(b.x, b.y, b.w, b.h);
    }
    c.fillStyle = '#a29bfe';
    c.fillRect(this.paddle.x, this.height - this.paddle.h, this.paddle.w, this.pad
dle.h);
    c.fillStyle = '#fff';
    c.beginPath();
    c.arc(this.ball.x, this.ball.y, this.ball.r, 0, Math.PI * 2);
    c.fill();
  },

  updateStats() {
    if (this.statsCb) {
      this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, '_—R ': this.score, 'u T}
': this.lives });
    }
  },

  gameOver(won = false) {
    this.running = false;
    clearInterval(this.loop);
    document.removeEventListener('keydown', this.handleKey);
    document.removeEventListener('mousemove', this.handleMouse);
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = won
      ? `<h3>Ø<ß‰ `mUœ• Qsÿ </h3><p>• •ÇN†Qh•è ${this.maxLevel} Qsÿ _—R : ${this.score}</p><button
 class="btn" id="replayBtn">Q•geN \@</button>`
      : `<h3>n8b ~Óg_</h3><p>{, ${this.level}/${this.maxLevel} Qs | _—R : ${this.score}</p
><button class="btn" id="replayBtn">Q•geN \@</button>`;
```

```
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.destroy();
      this.init(this.container, this.statsCb);
    });
  },

  destroy() {
    this.running = false;
    clearInterval(this.loop);
    document.removeEventListener('keydown', this.handleKey);
    document.removeEventListener('mousemove', this.handleMouse);
  }
};


// === Bubble Shooter (js/games/bubble.js) ===

/**
 * Bubble Shooter (lálá\ bK) - 5 Qs•ïQsj!_
 * @author ch2077
 * @version 3.1.0
 * @date 2025-04-28 [ŒQh‘ÍQ™S9‘M•;•‘ÿ e9N:•Qh<•»c¥
 * @date 2025-05-08 OîY spawnBubble˜œ,rN W(RiOYlálàN-v„bug
 *
 * ~ÏQxlálàŸ™s©lÕÿ hex grid^ \@ÿ PvepˆL•)•ÛSJN*h<[Pÿ
 * S9‘MhÀmKÿ flood fillc Qm‰Ò•»c¥b~T ,r•Þ• R ‘Ïÿ "e3mˆ–d
 * mnR¨lálàhÀmKÿ BFSNÎ˜vˆL•MS†ÿ N •Þ• v„c‰„=^vR R
 * Qs•.OîY ÿ spawnBubbles°W(SêNÎRiOYlálá˜œ,rl`N-–•g:
 */
const BubbleGame = {
  canvas: null, ctx: null,
  bubbles: [], shooter: { x: 0, y: 0 },
  currentBubble: null,
  // Qm,rlálàÿ c QsSa‰ã• ˜œ,repÿ 4!’4!’5!’5!’6ÿ
  allColors: ['#ff6b6b','#ffa726','#ffd93d','#00d2a0','#54a0ff','#a29bfe'],
  colors: [],
  score: 0, shots: 0,
  rows: 8, cols: 10,
  cellSize: 0, radius: 0,
  container: null, statsCb: null,
  animFrame: null, running: false,
  aimLine: { angle: Math.PI / 2 },
  nextId: 0,
  level: 1, maxLevel: 5,
  levelConfigs: [
    { rows: 5, numColors: 4 },
    { rows: 6, numColors: 4 },
    { rows: 7, numColors: 5 },
    { rows: 8, numColors: 5 },
    { rows: 9, numColors: 6 },
  ],

  init(container, statsCb) {
```

```
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0;
    this.shots = 0;
    this.level = 1;
    this.running = true;
    this.currentBubble = null;
    this.bubbles = [];
    this.nextId = 0;
    this.render();
    this.setupCanvas();
    this.showLevelIntro(() => {
      this.startLevel();
      this.loop();
    });
  },

  startLevel() {
    const cfg = this.levelConfigs[this.level - 1];
    this.colors = this.allColors.slice(0, cfg.numColors);
    this.initGrid(cfg.rows);
    this.spawnBubble();
    this.updateStats();
  },

  render() {
    this.container.innerHTML = `<canvas id="bubbleCanvas" style="width:100%;max-wi
dth:400px;border-radius:8px;touch-action:none"></canvas>`;
  },

  setupCanvas() {
    this.canvas = document.getElementById('bubbleCanvas');
    this.ctx = this.canvas.getContext('2d');
    const w = Math.min(400, this.container.clientWidth - 32);
    this.canvas.width = w;
    this.canvas.height = w * 1.5;
    this.cellSize = this.canvas.width / this.cols;
    this.radius = this.cellSize * 0.42;
    this.shooter = { x: this.canvas.width / 2, y: this.canvas.height - this.radius
 - 4 };

    const updateAim = (clientX, clientY) => {
      const rect = this.canvas.getBoundingClientRect();
      const mx = (clientX - rect.left) * (this.canvas.width / rect.width);
      const my = (clientY - rect.top) * (this.canvas.height / rect.height);
      const dx = mx - this.shooter.x;
      const dy = my - this.shooter.y;
      if (dy < 0) this.aimLine.angle = Math.atan2(-dy, dx);
    };

    this.canvas.addEventListener('mousemove', e => { updateAim(e.clientX, e.client
Y); });
    this.canvas.addEventListener('touchmove', e => { e.preventDefault(); updateAim
(e.touches[0].clientX, e.touches[0].clientY); });
    this.canvas.addEventListener('click', () => { this.shoot(); });
```

```
    this.canvas.addEventListener('touchend', () => { this.shoot(); });
  },

  initGrid(numRows) {
    this.bubbles = [];
    for (let r = 0; r < numRows; r++) {
      const offset = (r % 2) * this.cellSize / 2;
      const maxCols = (r % 2 === 1) ? this.cols - 1 : this.cols;
      for (let c = 0; c < maxCols; c++) {
        this.bubbles.push({
          id: this.nextId++,
          x: offset + c * this.cellSize + this.cellSize / 2,
          y: r * this.cellSize * 0.87 + this.cellSize / 2 + 10,
          r: this.radius,
          color: this.colors[Math.floor(Math.random() * this.colors.length)],
          row: r, col: c
        });
      }
    }
  },

  showLevelIntro(cb) {
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">lálá\ bK · ${this.levelConfigs[this.level-1].numColors} ,r</div>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    setTimeout(() => { if (div.parentNode) div.remove(); cb(); }, 1500);
  },

  spawnBubble() {
    // Only pick colors that still exist on the grid (or any if grid is empty)
    let pool = this.colors;
    if (this.bubbles.length > 0) {
      const remainingColors = [...new Set(this.bubbles.map(b => b.color))];
      if (remainingColors.length > 0) pool = remainingColors;
    }
    this.currentBubble = {
      x: this.shooter.x, y: this.shooter.y,
      r: this.radius,
      color: pool[Math.floor(Math.random() * pool.length)],
      vx: 0, vy: 0, active: true, shooting: false
    };
  },

  shoot() {
    if (!this.currentBubble || this.currentBubble.shooting) return;
    const speed = 14;
    this.currentBubble.vx = Math.cos(this.aimLine.angle) * speed;
    this.currentBubble.vy = -Math.sin(this.aimLine.angle) * speed;
    this.currentBubble.shooting = true;
    this.shots++;
  },
```

```
  update() {
    const cb = this.currentBubble;
    if (!cb || !cb.shooting) return;

    cb.x += cb.vx;
    cb.y += cb.vy;

    if (cb.x - cb.r <= 0 || cb.x + cb.r >= this.canvas.width) {
      cb.vx *= -1;
      cb.x = Math.max(cb.r, Math.min(this.canvas.width - cb.r, cb.x));
    }

    if (cb.y - cb.r <= 0) {
      this.snapBubble(cb);
      return;
    }

    for (let i = 0; i < this.bubbles.length; i++) {
      const b = this.bubbles[i];
      const dx = cb.x - b.x;
      const dy = cb.y - b.y;
      if (Math.sqrt(dx * dx + dy * dy) < this.radius * 2 * 0.95) {
        this.snapBubble(cb);
        return;
      }
    }

    if (cb.y > this.canvas.height + this.radius) {
      cb.shooting = false; cb.active = false;
      this.spawnBubble();
    }
  },

  snapBubble(bubble) {
    bubble.active = false; bubble.shooting = false; bubble.id = this.nextId++;

    let bestR = 0, bestC = 0, bestDist = Infinity;
    for (let r = 0; r < this.rows; r++) {
      const offset = (r % 2) * this.cellSize / 2;
      const maxCols = (r % 2 === 1) ? this.cols - 1 : this.cols;
      for (let c = 0; c < maxCols; c++) {
        const gx = offset + c * this.cellSize + this.cellSize / 2;
        const gy = r * this.cellSize * 0.87 + this.cellSize / 2 + 10;
        const dx = bubble.x - gx, dy = bubble.y - gy;
        const d = dx * dx + dy * dy;
        if (d < bestDist) {
          if (!this.bubbles.some(b => b.row === r && b.col === c)) {
            bestDist = d; bestR = r; bestC = c;
          }
        }
      }
    }

    bubble.row = bestR; bubble.col = bestC;
    const offset = (bestR % 2) * this.cellSize / 2;
```

```
    bubble.x = offset + bestC * this.cellSize + this.cellSize / 2;
    bubble.y = bestR * this.cellSize * 0.87 + this.cellSize / 2 + 10;
    this.bubbles.push(bubble);

    const matched = this.findMatches(bubble);
    if (matched.length >= 3) {
      this.score += matched.length * 10;
      const matchedIds = new Set(matched.map(b => b.id));
      this.bubbles = this.bubbles.filter(b => !matchedIds.has(b.id));
      this.removeFloating();
    }

    this.spawnBubble();
    this.updateStats();

    // Level clear or game over
    if (this.bubbles.length === 0) {
      if (this.level >= this.maxLevel) { this.gameOver(true); }
      else { this.levelClear(); }
      return;
    }
    const loseRow = this.bubbles.some(b => b.y + b.r > this.shooter.y - 10);
    if (loseRow) this.gameOver(false);
  },

  getNeighbors(bubble) {
    const r = bubble.row, c = bubble.col;
    const candidates = [];
    if (r % 2 === 0) {
      candidates.push({ row: r, col: c - 1 }, { row: r, col: c + 1 },
        { row: r - 1, col: c - 1 }, { row: r - 1, col: c },
        { row: r + 1, col: c - 1 }, { row: r + 1, col: c });
    } else {
      candidates.push({ row: r, col: c - 1 }, { row: r, col: c + 1 },
        { row: r - 1, col: c }, { row: r - 1, col: c + 1 },
        { row: r + 1, col: c }, { row: r + 1, col: c + 1 });
    }
    const neighbors = [];
    for (const pos of candidates) {
      const b = this.bubbles.find(b => b.row === pos.row && b.col === pos.col);
      if (b && b.color === bubble.color) neighbors.push(b);
    }
    return neighbors;
  },

  findMatches(bubble) {
    const visited = new Set(), matches = [], stack = [bubble];
    while (stack.length) {
      const b = stack.pop();
      if (visited.has(b.id)) continue;
      visited.add(b.id); matches.push(b);
      for (const nb of this.getNeighbors(b)) {
        if (!visited.has(nb.id)) stack.push(nb);
      }
    }
```

```
    return matches;
  },

  removeFloating() {
    const connected = new Set();
    const queue = this.bubbles.filter(b => b.row <= 0);
    queue.forEach(b => connected.add(b.id));
    while (queue.length) {
      const b = queue.shift();
      const r = b.row, c = b.col;
      const candidates = [];
      if (r % 2 === 0) {
        candidates.push({ row: r, col: c - 1 }, { row: r, col: c + 1 },
          { row: r - 1, col: c - 1 }, { row: r - 1, col: c },
          { row: r + 1, col: c - 1 }, { row: r + 1, col: c });
      } else {
        candidates.push({ row: r, col: c - 1 }, { row: r, col: c + 1 },
          { row: r - 1, col: c }, { row: r - 1, col: c + 1 },
          { row: r + 1, col: c }, { row: r + 1, col: c + 1 });
      }
      for (const pos of candidates) {
        const nb = this.bubbles.find(b => b.row === pos.row && b.col === pos.col);
        if (nb && !connected.has(nb.id)) { connected.add(nb.id); queue.push(nb); }
      }
    }
    const before = this.bubbles.length;
    this.bubbles = this.bubbles.filter(b => connected.has(b.id));
    this.score += (before - this.bubbles.length) * 5;
  },

  levelClear() {
    this.running = false;
    cancelAnimationFrame(this.animFrame);
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">'  {, ${this.level} Qs• •Çÿ </div><div class
="level-title">QÆY •ÛQe{, ${this.level + 1} Qs</div>`;
    this.container.appendChild(div);
    setTimeout(() => {
      if (div.parentNode) div.remove();
      this.level++;
      this.running = true;
      this.bubbles = [];
      this.currentBubble = null;
      this.showLevelIntro(() => {
        this.startLevel();
        this.loop();
      });
    }, 1800);
  },

  draw() {
    const ctx = this.ctx;
    ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
    for (const b of this.bubbles) this.drawBubble(b.x, b.y, b.r, b.color);
```

```
    if (this.currentBubble && !this.currentBubble.shooting) {
      ctx.strokeStyle = 'rgba(255,255,255,0.3)';
      ctx.lineWidth = 1; ctx.setLineDash([4, 4]);
      ctx.beginPath();
      ctx.moveTo(this.shooter.x, this.shooter.y);
      ctx.lineTo(this.shooter.x + Math.cos(this.aimLine.angle) * 80, this.shooter.
y - Math.sin(this.aimLine.angle) * 80);
      ctx.stroke(); ctx.setLineDash([]);
    }
    if (this.currentBubble) this.drawBubble(this.currentBubble.x, this.currentBubb
le.y, this.currentBubble.r, this.currentBubble.color);
  },

  drawBubble(x, y, r, color) {
    const ctx = this.ctx;
    ctx.beginPath(); ctx.arc(x, y, r, 0, Math.PI * 2);
    ctx.fillStyle = color; ctx.fill();
    ctx.strokeStyle = 'rgba(255,255,255,0.25)'; ctx.lineWidth = 1; ctx.stroke();
    ctx.beginPath(); ctx.arc(x - r * 0.3, y - r * 0.3, r * 0.3, 0, Math.PI * 2);
    ctx.fillStyle = 'rgba(255,255,255,0.3)'; ctx.fill();
  },

  loop() {
    if (!this.running) return;
    this.update(); this.draw();
    this.animFrame = requestAnimationFrame(() => this.loop());
  },

  updateStats() {
    if (this.statsCb) this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, '_—R ':
 this.score });
  },

  gameOver(won = false) {
    this.running = false;
    cancelAnimationFrame(this.animFrame);
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = won
      ? `<h3>Ø<ß‰ `mUœ• Qsÿ </h3><p>• •ÇN†Qh•è ${this.maxLevel} Qsÿ _—R : ${this.score}</p><button
 class="btn" id="replayBtn">Q•geN \@</button>`
      : `<h3>n8b ~Óg_</h3><p>{, ${this.level}/${this.maxLevel} Qs | _—R : ${this.score}</p
><button class="btn" id="replayBtn">Q•geN k!</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.destroy(); this.init(this.container, this.statsCb);
    });
  },

  destroy() {
    this.running = false;
    cancelAnimationFrame(this.animFrame);
  }
};
```

```
// === Space Invaders (js/games/invaders.js) ===

/**
 * Space Invaders (Y*zzOµue€ ) - 5 lâ•ïQsj!_
 * @author ch2077
 * @version 2.0.1
 * @date 2025-04-12 ‘ÍQ™N:5lâ•ïQsÿ kĬlâN T ,0– ‘M•n
 * @date 2025-04-25 R N†•ÇQsYVR±u T}g:R6ÿ • QsVÞná
 *
 * ~ĬQxˆWg:\ Qûÿ mˆpmb@g Y f ,0– •ÛQeN N lâ
 * Y f Nº• – ]æSóyûR¨‰æ•¹N yûÿ –•g:T s©[¶\ Qû
 * 5lâ–¾^¦• Xžÿ O¦[ß!’z•Qû!’|¾,ñ!’s‹rL!’BOSS
 * e/c •.vØe¹T •.+Ÿ h yûR¨+‰æ\ObÖR¨c§R6˜Þ,9
 */
const InvadersGame = {
  canvas: null, ctx: null,
  player: { x: 0, y: 0, w: 40, h: 30 },
  aliens: [], bullets: [], alienBullets: [],
  alienDir: 1, alienSpeed: 0.5, alienDrop: 8,
  score: 0, lives: 3, level: 1, maxLevel: 5,
  running: false, animFrame: null,
  container: null, statsCb: null,
  moveLeft: false, moveRight: false,
  lastShot: 0, shotCooldown: 400,
  transitioning: false,

  levelConfigs: [
    { rows: 3, speed: 0.5, shootRate: 0.01, name: 'O¦[ß,0– ' },
    { rows: 4, speed: 0.7, shootRate: 0.015, name: 'z•Qû,0– ' },
    { rows: 5, speed: 0.9, shootRate: 0.02, name: '|¾,ñ,0– ' },
    { rows: 5, speed: 1.1, shootRate: 0.025, name: 's‹rL,0– ' },
    { rows: 6, speed: 1.3, shootRate: 0.03, name: 'BOSS,0– ' },
  ],

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0; this.lives = 3; this.level = 1;
    this.bullets = []; this.alienBullets = [];
    this.alienDir = 1;
    this.moveLeft = false; this.moveRight = false;
    this.running = true; this.transitioning = false;
    this.render();
    this.setupCanvas();
    this.bindControls();
    this.showLevelIntro(() => {
      this.spawnAliens();
      this.updateStats();
      this.loop();
    });
  },

  render() {
```

```
    this.container.innerHTML = `<canvas id="invCanvas" style="width:100%;max-width
:400px;border-radius:8px;touch-action:none"></canvas>`;
  },

  setupCanvas() {
    this.canvas = document.getElementById('invCanvas');
    this.ctx = this.canvas.getContext('2d');
    const w = Math.min(400, this.container.clientWidth - 32);
    this.canvas.width = w;
    this.canvas.height = w * 1.4;
    this.player.x = this.canvas.width / 2;
    this.player.y = this.canvas.height - 50;
  },

  showLevelIntro(cb) {
    this.transitioning = true;
    const cfg = this.levelConfigs[this.level - 1];
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">${cfg.name}</div>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    setTimeout(() => {
      if (div.parentNode) div.remove();
      this.transitioning = false;
      cb();
    }, 1500);
  },

  spawnAliens() {
    const cfg = this.levelConfigs[this.level - 1];
    this.alienSpeed = cfg.speed;
    this.alienDir = 1;
    this.aliens = [];
    const cols = 8, rows = cfg.rows;
    const w = this.canvas.width / (cols + 1);
    for (let r = 0; r < rows; r++) {
      for (let c = 0; c < cols; c++) {
        this.aliens.push({
          x: w + c * w, y: 40 + r * 40,
          w: 24, h: 20, alive: true,
          type: r % 3
        });
      }
    }
  },

  bindControls() {
    const keydown = (e) => {
      if (e.key === 'ArrowLeft' || e.key === 'a') this.moveLeft = true;
      if (e.key === 'ArrowRight' || e.key === 'd') this.moveRight = true;
    };
    const keyup = (e) => {
      if (e.key === 'ArrowLeft' || e.key === 'a') this.moveLeft = false;
```

```
      if (e.key === 'ArrowRight' || e.key === 'd') this.moveRight = false;
    };
    document.addEventListener('keydown', keydown);
    document.addEventListener('keyup', keyup);
    this._keydown = keydown; this._keyup = keyup;

    this.canvas.addEventListener('touchmove', e => {
      e.preventDefault();
      const rect = this.canvas.getBoundingClientRect();
      const tx = (e.touches[0].clientX - rect.left) * (this.canvas.width / rect.wi
dth);
      this.player.x = Math.max(this.player.w/2, Math.min(this.canvas.width - this.
player.w/2, tx));
    });
    this.canvas.addEventListener('touchstart', e => {
      e.preventDefault();
      const rect = this.canvas.getBoundingClientRect();
      const tx = (e.touches[0].clientX - rect.left) * (this.canvas.width / rect.wi
dth);
      this.player.x = Math.max(this.player.w/2, Math.min(this.canvas.width - this.
player.w/2, tx));
      this.shoot();
    });
    this.canvas.addEventListener('click', () => this.shoot());
    this.canvas.addEventListener('mousemove', e => {
      const rect = this.canvas.getBoundingClientRect();
      const mx = (e.clientX - rect.left) * (this.canvas.width / rect.width);
      this.player.x = Math.max(this.player.w/2, Math.min(this.canvas.width - this.
player.w/2, mx));
    });
  },

  shoot() {
    const now = Date.now();
    if (now - this.lastShot < this.shotCooldown) return;
    this.lastShot = now;
    this.bullets.push({ x: this.player.x, y: this.player.y - 20, vy: -6 });
  },

  update() {
    if (this.transitioning) return;
    if (this.moveLeft) this.player.x -= 5;
    if (this.moveRight) this.player.x += 5;
    this.player.x = Math.max(this.player.w/2, Math.min(this.canvas.width - this.pl
ayer.w/2, this.player.x));

    const now = Date.now();
    if (now - this.lastShot > 800) this.shoot();

    for (const b of this.bullets) b.y += b.vy;
    this.bullets = this.bullets.filter(b => b.y > -10);

    for (const b of this.alienBullets) b.y += 2.5;
    this.alienBullets = this.alienBullets.filter(b => b.y < this.canvas.height + 1
0);
```

```
    const cfg = this.levelConfigs[this.level - 1];
    const liveAliens = this.aliens.filter(a => a.alive);
    if (liveAliens.length && Math.random() < cfg.shootRate) {
      const shooter = liveAliens[Math.floor(Math.random() * liveAliens.length)];
      this.alienBullets.push({ x: shooter.x, y: shooter.y + 10 });
    }

    let hitEdge = false;
    for (const a of this.aliens) {
      if (!a.alive) continue;
      a.x += this.alienDir * this.alienSpeed;
      if (a.x + a.w/2 >= this.canvas.width || a.x - a.w/2 <= 0) hitEdge = true;
    }
    if (hitEdge) {
      this.alienDir *= -1;
      for (const a of this.aliens) {
        if (!a.alive) continue;
        a.y += this.alienDrop;
        a.x += this.alienDir * this.alienSpeed;
      }
    }

    for (const b of this.bullets) {
      for (const a of this.aliens) {
        if (!a.alive) continue;
        if (Math.abs(b.x - a.x) < a.w/2 && Math.abs(b.y - a.y) < a.h/2) {
          a.alive = false; b.y = -999;
          this.score += (a.type + 1) * 10;
        }
      }
    }
    this.bullets = this.bullets.filter(b => b.y > -10 && b.y < this.canvas.height
+ 10);

    for (const b of this.alienBullets) {
      if (Math.abs(b.x - this.player.x) < this.player.w/2 && Math.abs(b.y - this.p
layer.y) < this.player.h/2) {
        b.y = -999; this.lives--;
        if (this.lives <= 0) { this.gameOver(); return; }
      }
    }
    this.alienBullets = this.alienBullets.filter(b => b.y > -10);

    if (this.aliens.some(a => a.alive && a.y + a.h/2 >= this.player.y)) {
      this.lives = 0; this.gameOver(); return;
    }

    if (this.aliens.every(a => !a.alive)) {
      if (this.level >= this.maxLevel) { this.gameOver(true); return; }
      this.level++;
      this.lives = Math.min(5, this.lives + 1); // Bonus life
      this.updateStats();
      this.showLevelIntro(() => {
        this.spawnAliens();
```

```
      });
    }

    this.updateStats();
  },

  draw() {
    const ctx = this.ctx;
    ctx.clearRect(0, 0, this.canvas.width, this.canvas.height);
    for (const a of this.aliens) {
      if (!a.alive) continue;
      const colors = ['#54a0ff','#ffa726','#ff6b6b'];
      ctx.fillStyle = colors[a.type];
      ctx.fillRect(a.x - a.w/2, a.y - a.h/2, a.w, a.h);
      ctx.fillStyle = '#fff';
      ctx.fillRect(a.x - 5, a.y - 5, 3, 4);
      ctx.fillRect(a.x + 2, a.y - 5, 3, 4);
    }
    ctx.fillStyle = '#00d2a0';
    ctx.beginPath();
    ctx.moveTo(this.player.x, this.player.y - this.player.h/2);
    ctx.lineTo(this.player.x - this.player.w/2, this.player.y + this.player.h/2);
    ctx.lineTo(this.player.x + this.player.w/2, this.player.y + this.player.h/2);
    ctx.closePath(); ctx.fill();
    ctx.fillStyle = '#ffd93d';
    for (const b of this.bullets) ctx.fillRect(b.x - 2, b.y - 6, 4, 10);
    ctx.fillStyle = '#ff6b6b';
    for (const b of this.alienBullets) ctx.fillRect(b.x - 2, b.y - 4, 4, 8);
    ctx.fillStyle = 'rgba(255,255,255,0.5)';
    const starSeed = 42;
    for (let i = 0; i < 40; i++) {
      const sx = ((i * 137 + starSeed) % this.canvas.width);
      const sy = ((i * 251 + this.score * 10) % this.canvas.height);
      ctx.fillRect(sx, sy, 1.5, 1.5);
    }
  },

  loop() {
    if (!this.running) return;
    this.update(); this.draw();
    this.animFrame = requestAnimationFrame(() => this.loop());
  },

  updateStats() {
    if (this.statsCb) this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, '_—R ':
this.score, 'u T}': this.lives });
  },

  gameOver(won = false) {
    this.running = false;
    cancelAnimationFrame(this.animFrame);
    document.removeEventListener('keydown', this._keydown);
    document.removeEventListener('keyup', this._keyup);
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
```

```
    div.innerHTML = won
      ? `<h3>Ø<ß‰ `mUœ• Qsÿ </h3><p>mˆpmN†Qh•è ${this.maxLevel} lâ,0– ÿ _—R : ${this.score}</p><butt
on class="btn" id="replayBtn">Q•geN \@</button>`
      : `<h3>n8b ~Óg_</h3><p>R0•¾{, ${this.level}/${this.maxLevel} Qs | _—R : ${this.score}<
/p><button class="btn" id="replayBtn">Q•geN k!</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.destroy(); this.init(this.container, this.statsCb);
    });
  },

  destroy() {
    this.running = false;
    cancelAnimationFrame(this.animFrame);
    document.removeEventListener('keydown', this._keydown);
    document.removeEventListener('keyup', this._keyup);
  }
};


// === Snake (js/games/snake.js) ===

/**
 * Snake Game (•*T †Ç) - 5 Qs•ïQsj!_
 * @author ch2077
 * @version 2.1.0
 * @date 2025-03-15 ‘Íg„yûR¨zï• ‘M
 * @date 2025-04-20 XžR •ïQs|û~ß
 *
 * ~ÏQx•*T †Ç[žs°ÿ ‘Çu(tile-based•Qh<yûR¨
 * yûR¨zïPZN†swipebKR¿ + D-PadSÌ• ‘Mÿ –2kbe¹T ‹ïR$R N†15pxk{S:
 * • ^¦–•QsSa• Xžÿ R YË100ms/keÿ kÏQs–MON~¦15-20ms
 */
const SnakeGame = {
  canvas: null, ctx: null,
  tileCount: 20, tileSize: 0,  // 20x20h<ÿ W0Vþ\:[ø•ê• ^”
  snake: [], food: null,
  dir: { x: 1, y: 0 }, nextDir: { x: 1, y: 0 },
  // lèa ÿ nextDiru(NŽ• Q²•“Qeÿ –2kbT N ^'Q…N$k!SÍT [ü•ô•êdž
  score: 0, speed: 100,
  loop: null, running: false,
  container: null, statsCb: null,
  touchStart: null,
  level: 1, maxLevel: 5,
  targetScore: 50,
  levelConfigs: [
    { target: 50, speed: 100, name: 'e°bKQe•è' },
    { target: 100, speed: 85, name: 'n QeOsXƒ' },
    { target: 150, speed: 70, name: '• ^¦c b ' },
    { target: 200, speed: 55, name: 'g•–PSÍ^”' },
    { target: 300, speed: 45, name: '†Çs‹NK•ï' },
  ],

  init(container, statsCb) {
```

```
    this.container = container;
    this.statsCb = statsCb;
    this.snake = [{ x: 10, y: 10 }];
    this.dir = { x: 1, y: 0 }; this.nextDir = { x: 1, y: 0 };
    this.score = 0; this.level = 1;
    const cfg = this.levelConfigs[0];
    this.speed = cfg.speed; this.targetScore = cfg.target;
    this.running = true; this.touchStart = null;

    const size = Math.min(400, this.container.clientWidth - 48);
    this.canvas = document.createElement('canvas');
    this.canvas.className = 'game-canvas';
    this.canvas.width = size; this.canvas.height = size;
    this.ctx = this.canvas.getContext('2d');
    this.tileSize = size / this.tileCount;

    this.container.innerHTML = '';
    this.container.appendChild(this.canvas);
    this.container.appendChild(this.buildDPad());
    this.spawnFood();
    this.showLevelIntro(() => {
      this.updateStats();
      document.addEventListener('keydown', this._handleKey);
      this.canvas.addEventListener('touchstart', this._touchStart, {passive:false}
);
      this.canvas.addEventListener('touchend', this._touchEnd, {passive:false});
      this.loop = setInterval(() => this.tick(), this.speed);
    });
  },

  showLevelIntro(cb) {
    this.running = false;
    const cfg = this.levelConfigs[this.level - 1];
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">{, ${this.level} Qs</div><div class="lev
el-title">${cfg.name} · vîh  ${cfg.target} R </div>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    setTimeout(() => {
      if (div.parentNode) div.remove();
      this.running = true;
      cb();
    }, 1500);
  },

  _handleKey(e) {
    const k = e.key.toLowerCase();
    if (k === 'arrowup' || k === 'w') SnakeGame.setDir(0, -1);
    else if (k === 'arrowdown' || k === 's') SnakeGame.setDir(0, 1);
    else if (k === 'arrowleft' || k === 'a') SnakeGame.setDir(-1, 0);
    else if (k === 'arrowright' || k === 'd') SnakeGame.setDir(1, 0);
  },

  setDir(dx, dy) {
```

```
    if (this.dir.x === -dx && this.dir.y === -dy) return;
    this.nextDir = { x: dx, y: dy };
  },

  _touchStart(e) { e.preventDefault(); if(e.touches[0]) SnakeGame.touchStart={x:e.
touches[0].clientX,y:e.touches[0].clientY}; },
  _touchEnd(e) {
    e.preventDefault();
    if (!SnakeGame.touchStart) return;
    const dx = e.changedTouches[0].clientX - SnakeGame.touchStart.x;
    const dy = e.changedTouches[0].clientY - SnakeGame.touchStart.y;
    SnakeGame.touchStart = null;
    if (Math.abs(dx) < 15 && Math.abs(dy) < 15) return;
    if (Math.abs(dx) > Math.abs(dy)) SnakeGame.setDir(dx > 0 ? 1 : -1, 0);
    else SnakeGame.setDir(0, dy > 0 ? 1 : -1);
  },

  buildDPad() {
    const d = document.createElement('div');
    d.className = 'dpad';
    d.innerHTML = `
      <button class="dpad-btn dpad-up" id="dpUp">%²</button>
      <button class="dpad-btn dpad-left" id="dpLeft">%À</button>
      <button class="dpad-btn dpad-right" id="dpRight">%¶</button>
      <button class="dpad-btn dpad-down" id="dpDown">%¼</button>`;
    setTimeout(() => {
      const bind = (id, dx, dy) => {
        const el = document.getElementById(id);
        if (el) el.addEventListener('touchstart', e => { e.preventDefault(); Snake
Game.setDir(dx, dy); });
      };
      bind('dpUp', 0, -1); bind('dpDown', 0, 1); bind('dpLeft', -1, 0); bind('dpRi
ght', 1, 0);
    }, 0);
    return d;
  },

  tick() {
    if (!this.running) return;
    this.dir = this.nextDir;
    const head = this.snake[0];
    const newHead = { x: head.x + this.dir.x, y: head.y + this.dir.y };
    if (newHead.x < 0 || newHead.x >= this.tileCount || newHead.y < 0 || newHead.y
 >= this.tileCount) { this.gameOver(); return; }
    if (this.snake.some(s => s.x === newHead.x && s.y === newHead.y)) { this.gameO
ver(); return; }
    this.snake.unshift(newHead);
    if (newHead.x === this.food.x && newHead.y === this.food.y) {
      this.score += 10; this.updateStats();
      if (this.score >= this.targetScore) {
        if (this.level >= this.maxLevel) { this.gameOver(true); return; }
        this.levelUp();
        return;
      }
      this.spawnFood();
```

```
      if (this.speed > 35) { this.speed -= 2; clearInterval(this.loop); this.loop
= setInterval(() => this.tick(), this.speed); }
    } else { this.snake.pop(); }
    this.draw();
  },

  spawnFood() {
    do { this.food = { x: Math.floor(Math.random() * this.tileCount), y: Math.floo
r(Math.random() * this.tileCount) }; }
    while (this.snake.some(s => s.x === this.food.x && s.y === this.food.y));
  },

  levelUp() {
    clearInterval(this.loop);
    this.running = false;
    const div = document.createElement('div');
    div.className = 'level-intro';
    div.innerHTML = `<div class="level-num">'  {, ${this.level} Qs• •Çÿ </div><div class
="level-title">QÆY •ÛQe{, ${this.level + 1} Qs</div>`;
    this.container.appendChild(div);
    setTimeout(() => {
      if (div.parentNode) div.remove();
      this.level++;
      const cfg = this.levelConfigs[this.level - 1];
      this.speed = cfg.speed; this.targetScore = cfg.target;
      this.snake = [{ x: 10, y: 10 }];
      this.dir = { x: 1, y: 0 }; this.nextDir = { x: 1, y: 0 };
      this.spawnFood();
      this.showLevelIntro(() => {
        this.updateStats();
        this.loop = setInterval(() => this.tick(), this.speed);
      });
    }, 1800);
  },

  draw() {
    const c = this.ctx, t = this.tileSize;
    c.clearRect(0, 0, this.canvas.width, this.canvas.height);
    c.fillStyle = '#ff6b6b';
    c.fillRect(this.food.x * t + 2, this.food.y * t + 2, t - 4, t - 4);
    this.snake.forEach((s, i) => {
      c.fillStyle = i === 0 ? '#a29bfe' : '#6c5ce7';
      c.fillRect(s.x * t + 1, s.y * t + 1, t - 2, t - 2);
    });
  },

  updateStats() {
    if (this.statsCb) this.statsCb({ 'QsSa': `${this.level}/${this.maxLevel}`, '_—R ':
 `${this.score}/${this.targetScore}`, '••^¦': this.snake.length });
  },

  gameOver(won = false) {
    this.running = false; clearInterval(this.loop);
    document.removeEventListener('keydown', this._handleKey);
    const div = document.createElement('div');
```

```
    div.className = 'game-over-overlay';
    div.innerHTML = won
      ? `<h3>Ø<ß‰ `mUœ• Qsÿ </h3><p>• •ÇN†Qh•è ${this.maxLevel} Qsÿ g ~È_—R : ${this.score}</p><butt
on class="btn" id="replayBtn">Q•geN \@</button>`
      : `<h3>n8b ~Óg_ÿ </h3><p>{, ${this.level}/${this.maxLevel} Qs | _—R : ${this.score}/$
{this.targetScore} | ••^¦: ${this.snake.length}</p><button class="btn" id="replayBtn
">Q•geN \@</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => { this.de
stroy(); this.init(this.container, this.statsCb); });
  },

  destroy() { this.running = false; clearInterval(this.loop); document.removeEvent
Listener('keydown', this._handleKey); }
};


// === 2048 (js/games/2048.js) ===

/**
 * 2048 Game
 * @author ch2077
 * @version 1.5.3
 * @date 2025-03-08 OîY nÑR¨Y1eH•î˜˜
 * @date 2025-05-02 XžR Ÿ h bÖbýe/c (hL—bzïN_€ýbÖN†)
 *
 * ~ÏQx2048ep[WT ^vÿ 4x4•Qh<
 * nÑR¨{—lÕÿ kÏˆL/R rìzËslideÿ QHnä0Q•T ^vvø•»vø{Iÿ g+\>ˆe0
 * yûR¨zïu(touch start/endWPh ]îR$e-e¹T ÿ – P<20px••QM‹ï‰æ
 * desktopzïR N†mousedown/mouseupbÖbýÿ – P<15px
 */
const Game2048 = {
  grid: [],       // 4x4ep[Wwé–5
  score: 0,       // _SRM_—R ÿ T ^veö}/R ÿ
  container: null,
  statsCb: null,
  boardEl: null,
  touchStart: null,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0;
    this.touchStart = null;
    this.grid = Array.from({ length: 4 }, () => Array(4).fill(0));
    this.render();
    this.spawn();
    this.spawn();
    this.updateBoard();
    this.updateStats();
    document.addEventListener('keydown', this._handleKey);
    this.boardEl.addEventListener('touchstart', this._touchStart, {passive:false})
;
    this.boardEl.addEventListener('touchend', this._touchEnd, {passive:false});
```

```
    this.boardEl.addEventListener('mousedown', this._mouseDown);
    this.boardEl.addEventListener('mouseup', this._mouseUp);
  },

  render() {
    this.container.innerHTML = '<div class="game-2048"><div class="board-2048"></d
iv></div>';
    this.boardEl = this.container.querySelector('.board-2048');
    for (let i = 0; i < 16; i++) {
      const cell = document.createElement('div');
      cell.className = 'cell-2048';
      this.boardEl.appendChild(cell);
    }
  },

  spawn() {
    const empty = [];
    for (let r = 0; r < 4; r++)
      for (let c = 0; c < 4; c++)
        if (this.grid[r][c] === 0) empty.push({ r, c });
    if (empty.length === 0) return;
    const { r, c } = empty[Math.floor(Math.random() * empty.length)];
    this.grid[r][c] = Math.random() < 0.9 ? 2 : 4;
  },

  updateBoard() {
    const cells = this.boardEl.querySelectorAll('.cell-2048');
    cells.forEach((cell, i) => {
      const r = Math.floor(i / 4);
      const c = i % 4;
      const v = this.grid[r][c];
      cell.textContent = v || '';
      cell.className = 'cell-2048' + (v ? ` tile-${v}` : '');
    });
  },

  updateStats() {
    if (this.statsCb) this.statsCb({ '_—R ': this.score });
  },

  doMove(dir) {
    let moved = false;
    const old = this.grid.map(r => [...r]);
    if (dir === 'up') moved = this.moveUp();
    else if (dir === 'down') moved = this.moveDown();
    else if (dir === 'left') moved = this.moveLeft();
    else if (dir === 'right') moved = this.moveRight();
    else return;
    if (moved) {
      this.spawn();
      this.updateBoard();
      this.updateStats();
      if (this.checkWin()) {
        this.showResult('Ø<ß‰ O`•bN†ÿ •¾R0 2048ÿ ');
      } else if (this.checkLose()) {
```

```
        this.showResult('n8b ~Óg_ÿ l¡g SïyûR¨v„N†');
      }
    }
  },

  _handleKey(e) {
    const map = { ArrowUp:'up', ArrowDown:'down', ArrowLeft:'left', ArrowRight:'ri
ght' };
    if (map[e.key]) { e.preventDefault(); Game2048.doMove(map[e.key]); }
  },

  _touchStart(e) { e.preventDefault(); Game2048.touchStart = { x: e.touches[0].cli
entX, y: e.touches[0].clientY }; },
  _touchEnd(e) {
    e.preventDefault();
    if (!Game2048.touchStart) return;
    const dx = e.changedTouches[0].clientX - Game2048.touchStart.x;
    const dy = e.changedTouches[0].clientY - Game2048.touchStart.y;
    Game2048.touchStart = null;
    if (Math.abs(dx) < 20 && Math.abs(dy) < 20) return;
    if (Math.abs(dx) > Math.abs(dy)) {
      Game2048.doMove(dx > 0 ? 'right' : 'left');
    } else {
      Game2048.doMove(dy > 0 ? 'down' : 'up');
    }
  },

  _mouseDown(e) { Game2048.touchStart = { x: e.clientX, y: e.clientY }; },
  _mouseUp(e) {
    if (!Game2048.touchStart) return;
    const dx = e.clientX - Game2048.touchStart.x;
    const dy = e.clientY - Game2048.touchStart.y;
    Game2048.touchStart = null;
    if (Math.abs(dx) < 15 && Math.abs(dy) < 15) return;
    if (Math.abs(dx) > Math.abs(dy)) {
      Game2048.doMove(dx > 0 ? 'right' : 'left');
    } else {
      Game2048.doMove(dy > 0 ? 'down' : 'up');
    }
  },

  slide(arr) {
    let filtered = arr.filter(v => v !== 0);
    for (let i = 0; i < filtered.length - 1; i++) {
      if (filtered[i] === filtered[i + 1]) {
        filtered[i] *= 2;
        this.score += filtered[i];
        filtered.splice(i + 1, 1);
      }
    }
    while (filtered.length < 4) filtered.push(0);
    return filtered;
  },

  moveLeft() { let moved=false; for(let r=0;r<4;r++){const old=[...this.grid[r]];t
```

```
  moveRight() { let moved=false; for(let r=0;r<4;r++){const old=[...this.grid[r]];
const rev=this.slide([...old].reverse()).reverse();this.grid[r]=rev;if(old.join()!
==rev.join())moved=true;} return moved; },
  moveUp() { let moved=false; for(let c=0;c<4;c++){const old=[this.grid[0][c],this
.grid[1][c],this.grid[2][c],this.grid[3][c]];const slided=this.slide(old);for(let
r=0;r<4;r++){if(this.grid[r][c]!==slided[r])moved=true;this.grid[r][c]=slided[r];}
} return moved; },
  moveDown() { let moved=false; for(let c=0;c<4;c++){const old=[this.grid[0][c],th
is.grid[1][c],this.grid[2][c],this.grid[3][c]];const slided=this.slide([...old].re
verse()).reverse();for(let r=0;r<4;r++){if(this.grid[r][c]!==slided[r])moved=true;
this.grid[r][c]=slided[r];}} return moved; },

  checkWin() { return this.grid.some(r => r.some(c => c === 2048)); },
  checkLose() {
    for (let r = 0; r < 4; r++)
      for (let c = 0; c < 4; c++) {
        if (this.grid[r][c] === 0) return false;
        if (c < 3 && this.grid[r][c] === this.grid[r][c + 1]) return false;
        if (r < 3 && this.grid[r][c] === this.grid[r + 1][c]) return false;
      }
    return true;
  },

  showResult(msg) {
    document.removeEventListener('keydown', this._handleKey);
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `<h3>${msg}</h3><p>g ~È_—R : ${this.score}</p><button class="btn"
id="replayBtn">Q•geN \@</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => { this.de
stroy(); this.init(this.container, this.statsCb); });
  },

  destroy() {
    document.removeEventListener('keydown', this._handleKey);
  }
};


// === Tetris (js/games/tetris.js) ===

/**
 * Tetris Game (OÄ•We¯e¹WW)
 * @author ch2077
 * @version 2.0.2
 * @date 2025-02-28 OíY eË•lx°džhÀmKbug
 * @date 2025-04-22 e°Xž‰æ\Oc ”®dÍc§
 *
 * ~ÏQxOÄ•We¯e¹WW[žs°ÿ 7yÍh QÆtetromino
 * eË•l{—lÕÿ wé–5•l•nT l4^s•û•lÿ ˜zeö”^90^¦ÿ
 * lèa ÿ Ie¹WW(••ga)eË•leö[¹f Sa•¹uLÿ R N†X™Ž"•;•‘
 */
const TetrisGame = {
```

```
  cols: 10, rows: 20, board: [],  // 10x20h QÆY'\
  canvas: null, ctx: null, tileSize: 24, // kĬh<24px
  piece: null, piecePos: { x: 0, y: 0 },

  // 7yÍh QÆe¹WW[šNIÿ u(1/0wé–5ˆhy:_br¶
  // ˜œ,rSÂ€ NESrHOÄ•We¯e¹WW‘M,r
  score: 0, level: 1, lines: 0, speed: 500,
  loop: null, running: false,
  container: null, statsCb: null,
  touchTimeout: null,

  pieces: [
    { shape: [[1,1,1,1]], color: '#54a0ff' },
    { shape: [[1,1],[1,1]], color: '#ffd93d' },
    { shape: [[0,1,0],[1,1,1]], color: '#a29bfe' },
    { shape: [[1,0,0],[1,1,1]], color: '#54a0ff' },
    { shape: [[0,0,1],[1,1,1]], color: '#ffa726' },
    { shape: [[0,1,1],[1,1,0]], color: '#00d2a0' },
    { shape: [[1,1,0],[0,1,1]], color: '#ff6b6b' },
  ],

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.score = 0; this.level = 1; this.lines = 0; this.speed = 500;
    this.running = true; this.touchTimeout = null;
    this.board = Array.from({ length: this.rows }, () => Array(this.cols).fill(0))
;

    const width = this.cols * this.tileSize + 2;
    const height = this.rows * this.tileSize + 2;
    this.canvas = document.createElement('canvas');
    this.canvas.className = 'game-canvas';
    this.canvas.width = width; this.canvas.height = height;
    this.ctx = this.canvas.getContext('2d');

    container.innerHTML = '';
    container.appendChild(this.canvas);
    container.appendChild(this.buildControls());
    this.spawnPiece();
    this.updateStats();
    document.addEventListener('keydown', this._handleKey);
    this.loop = setInterval(() => this.tick(), this.speed);
  },

  buildControls() {
    const d = document.createElement('div');
    d.className = 'tetris-controls';
    d.innerHTML = `
      <button class="btn small tet-btn" id="tetLeft">%À</button>
      <button class="btn small tet-btn" id="tetRotate">!»</button>
      <button class="btn small tet-btn" id="tetRight">%¶</button>
      <button class="btn small tet-btn" id="tetDrop" style="flex:2">%¼ N „=</button>`
;
    setTimeout(() => {
```

```
      ['tetLeft','tetRight','tetRotate','tetDrop'].forEach(id => {
        const el = document.getElementById(id);
        if (!el) return;
        el.addEventListener('touchstart', e => { e.preventDefault(); TetrisGame.ha
ndleTouch(id); });
        el.addEventListener('mousedown', e => { e.preventDefault(); TetrisGame.han
dleTouch(id); });
      });
    }, 0);
    return d;
  },

  handleTouch(id) {
    if (!this.running) return;
    if (id === 'tetLeft') { this.move(-1); this.draw(); }
    else if (id === 'tetRight') { this.move(1); this.draw(); }
    else if (id === 'tetRotate') { this.rotate(); this.draw(); }
    else if (id === 'tetDrop') { this.hardDrop(); }
  },

  spawnPiece() {
    const p = this.pieces[Math.floor(Math.random() * this.pieces.length)];
    this.piece = { shape: p.shape.map(r => [...r]), color: p.color };
    this.piecePos = { x: Math.floor((this.cols - this.piece.shape[0].length) / 2),
 y: 0 };
    if (this.collides(this.piece.shape, this.piecePos)) this.gameOver();
  },

  collides(shape, pos) {
    for (let r = 0; r < shape.length; r++)
      for (let c = 0; c < shape[r].length; c++) {
        if (!shape[r][c]) continue;
        const bx = pos.x + c, by = pos.y + r;
        if (bx < 0 || bx >= this.cols || by >= this.rows) return true;
        if (by >= 0 && this.board[by][bx]) return true;
      }
    return false;
  },

  place() {
    const { shape, color } = this.piece;
    for (let r = 0; r < shape.length; r++)
      for (let c = 0; c < shape[r].length; c++) {
        if (!shape[r][c]) continue;
        const bx = this.piecePos.x + c, by = this.piecePos.y + r;
        if (by < 0) { this.gameOver(); return; }
        this.board[by][bx] = color;
      }
    this.clearLines();
    this.spawnPiece();
  },

  clearLines() {
    let cleared = 0;
    for (let r = this.rows - 1; r >= 0; r--) {
```

```
      if (this.board[r].every(c => c !== 0)) {
        this.board.splice(r, 1); this.board.unshift(Array(this.cols).fill(0));
        cleared++; r++;
      }
    }
    if (cleared > 0) {
      this.lines += cleared;
      this.score += [0, 100, 300, 500, 800][cleared] * this.level;
      this.level = Math.floor(this.lines / 10) + 1;
      this.speed = Math.max(50, 500 - (this.level - 1) * 40);
      clearInterval(this.loop);
      this.loop = setInterval(() => this.tick(), this.speed);
      this.updateStats();
    }
  },

  tick() {
    if (!this.running) return;
    const np = { x: this.piecePos.x, y: this.piecePos.y + 1 };
    if (this.collides(this.piece.shape, np)) this.place();
    else this.piecePos = np;
    this.draw();
  },

  rotate() {
    const shape = this.piece.shape;
    const rot = shape[0].map((_, i) => shape.map(r => r[i]).reverse());
    if (!this.collides(rot, this.piecePos)) this.piece.shape = rot;
  },

  move(dx) {
    const np = { x: this.piecePos.x + dx, y: this.piecePos.y };
    if (!this.collides(this.piece.shape, np)) this.piecePos = np;
  },

  hardDrop() {
    while (!this.collides(this.piece.shape, { x: this.piecePos.x, y: this.piecePos
.y + 1 })) this.piecePos.y++;
    this.place(); this.draw();
  },

  _handleKey(e) {
    if (!TetrisGame.running) return;
    if (e.key === 'ArrowLeft') { TetrisGame.move(-1); TetrisGame.draw(); }
    else if (e.key === 'ArrowRight') { TetrisGame.move(1); TetrisGame.draw(); }
    else if (e.key === 'ArrowDown') TetrisGame.tick();
    else if (e.key === 'ArrowUp') { TetrisGame.rotate(); TetrisGame.draw(); }
    else if (e.key === ' ') { e.preventDefault(); TetrisGame.hardDrop(); }
  },

  draw() {
    const c = this.ctx, t = this.tileSize;
    c.clearRect(0, 0, this.canvas.width, this.canvas.height);
    for (let r = 0; r < this.rows; r++)
      for (let col = 0; col < this.cols; col++) {
```

```
        if (this.board[r][col]) { c.fillStyle = this.board[r][col]; c.fillRect(col
 * t, r * t, t - 1, t - 1); }
      }
    if (this.piece && this.running) {
      c.fillStyle = this.piece.color;
      for (let r = 0; r < this.piece.shape.length; r++)
        for (let col = 0; col < this.piece.shape[r].length; col++) {
          if (this.piece.shape[r][col]) c.fillRect((this.piecePos.x + col) * t, (t
his.piecePos.y + r) * t, t - 1, t - 1);
        }
    }
  },

  updateStats() { if (this.statsCb) this.statsCb({ '_—R ': this.score, '{I~§': this.le
vel, '^Lep': this.lines }); },

  gameOver() {
    this.running = false; clearInterval(this.loop);
    document.removeEventListener('keydown', this._handleKey);
    this.draw();
    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `<h3>n8b ~Óg_ÿ </h3><p>_—R : ${this.score} | {I~§: ${this.level} | m^–d:
${this.lines}^L</p><button class="btn" id="replayBtn">Q•geN \@</button>`;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => { this.de
stroy(); this.init(this.container, this.statsCb); });
  },

  destroy() { this.running = false; clearInterval(this.loop); document.removeEvent
Listener('keydown', this._handleKey); }
};


// === Minesweeper (js/games/minesweeper.js) ===

/**
 * Minesweeper Game (bk–÷)
 * @author ch2077
 * @version 1.3.1
 * @date 2025-01-20 R rH
 * @date 2025-03-12 OíY SÌQûep[W•êR¨cí_ v„bug
 *
 * ~ÏQxbk–÷ÿ 9x9R ~§–¾^¦ÿ 10N*–÷
 * ]æ•.•û_ ÿ Só•.cÒe×ÿ SÌQûep[W•êR¨cí_ ThVôÿ — ná•³e×epgaNöÿ
 * ™–k!p¹QûOÝ‹ÁN Ž)–÷ÿ [‰QhS:u b •;•’‰ÁinitGridÿ
 */
const MinesweeperGame = {
  rows: 9,     // ^Lep
  cols: 9,     // R ep
  mines: 10,   // –÷epÿ h QÆR ~§–¾^¦ÿ
  grid: [],
  revealed: [],
  flagged: [],
```

```
  container: null,
  statsCb: null,
  gameOver: false,

  init(container, statsCb) {
    this.container = container;
    this.statsCb = statsCb;
    this.gameOver = false;
    this.revealed = Array.from({ length: this.rows }, () => Array(this.cols).fill(
false));
    this.flagged = Array.from({ length: this.rows }, () => Array(this.cols).fill(f
alse));
    this.grid = Array.from({ length: this.rows }, () => Array(this.cols).fill(0));

    // Place mines
    let placed = 0;
    while (placed < this.mines) {
      const r = Math.floor(Math.random() * this.rows);
      const c = Math.floor(Math.random() * this.cols);
      if (this.grid[r][c] !== 'M') {
        this.grid[r][c] = 'M';
        placed++;
      }
    }

    // Calculate numbers
    for (let r = 0; r < this.rows; r++)
      for (let c = 0; c < this.cols; c++)
        if (this.grid[r][c] !== 'M') {
          this.grid[r][c] = this.countMines(r, c);
        }

    this.render();
    this.updateStats();
  },

  countMines(r, c) {
    let count = 0;
    for (let dr = -1; dr <= 1; dr++)
      for (let dc = -1; dc <= 1; dc++) {
        const nr = r + dr, nc = c + dc;
        if (nr >= 0 && nr < this.rows && nc >= 0 && nc < this.cols && this.grid[nr
][nc] === 'M')
          count++;
      }
    return count;
  },

  render() {
    this.container.innerHTML = '';
    const gridEl = document.createElement('div');
    gridEl.className = 'mine-grid';
    gridEl.style.gridTemplateColumns = `repeat(${this.cols}, 32px)`;
    gridEl.style.gridTemplateRows = `repeat(${this.rows}, 32px)`;
```

```
    for (let r = 0; r < this.rows; r++)
      for (let c = 0; c < this.cols; c++) {
        const cell = document.createElement('div');
        cell.className = 'mine-cell';
        cell.dataset.r = r;
        cell.dataset.c = c;
        cell.addEventListener('click', (e) => this.reveal(r, c, cell));
        cell.addEventListener('contextmenu', (e) => {
          e.preventDefault();
          this.toggleFlag(r, c, cell);
        });
        gridEl.appendChild(cell);
      }

    this.container.appendChild(gridEl);
  },

  reveal(r, c, cell) {
    if (this.gameOver) return;
    if (this.revealed[r][c] || this.flagged[r][c]) return;

    this.revealed[r][c] = true;

    if (this.grid[r][c] === 'M') {
      cell.classList.add('mine');
      cell.textContent = 'Ø=Ü£';
      this.showResult(false);
      return;
    }

    cell.classList.add('revealed');
    const v = this.grid[r][c];
    if (v > 0) {
      cell.textContent = v;
      cell.style.color = ['','#54a0ff','#00d2a0','#ff6b6b','#a29bfe','#ffa726','#f
fd93d','#e0e0f0','#8888aa'][v];
    } else {
      // Flood fill
      for (let dr = -1; dr <= 1; dr++)
        for (let dc = -1; dc <= 1; dc++) {
          const nr = r + dr, nc = c + dc;
          if (nr >= 0 && nr < this.rows && nc >= 0 && nc < this.cols && !this.reve
aled[nr][nc]) {
            const idx = nr * this.cols + nc;
            const ncell = this.container.querySelectorAll('.mine-cell')[idx];
            this.reveal(nr, nc, ncell);
          }
        }
    }

    if (this.checkWin()) {
      this.showResult(true);
    }
  },
```

```
  toggleFlag(r, c, cell) {
    if (this.gameOver || this.revealed[r][c]) return;
    this.flagged[r][c] = !this.flagged[r][c];
    if (this.flagged[r][c]) {
      cell.classList.add('flagged');
      cell.textContent = 'Ø=Þ©';
    } else {
      cell.classList.remove('flagged');
      cell.textContent = '';
    }
    this.updateStats();
  },

  checkWin() {
    for (let r = 0; r < this.rows; r++)
      for (let c = 0; c < this.cols; c++)
        if (this.grid[r][c] !== 'M' && !this.revealed[r][c]) return false;
    return true;
  },

  updateStats() {
    if (this.statsCb) {
      const flagged = this.flagged.flat().filter(Boolean).length;
      this.statsCb({ 'W0–÷': this.mines, 'h ‹°': flagged });
    }
  },

  showResult(won) {
    this.gameOver = true;
    // Reveal all
    const cells = this.container.querySelectorAll('.mine-cell');
    cells.forEach(cell => {
      const r = +cell.dataset.r, c = +cell.dataset.c;
      if (this.grid[r][c] === 'M') {
        cell.classList.add('mine');
        cell.textContent = 'Ø=Ü£';
      }
    });

    const div = document.createElement('div');
    div.className = 'game-over-overlay';
    div.innerHTML = `
      <h3>${won ? 'Ø<ß‰ O`•bN†ÿ ' : 'Ø=Ü¥ Ž)–÷N†ÿ '}</h3>
      <button class="btn" id="replayBtn">Q•geN \@</button>
    `;
    this.container.style.position = 'relative';
    this.container.appendChild(div);
    document.getElementById('replayBtn').addEventListener('click', () => {
      this.init(this.container, this.statsCb);
    });
  },

  destroy() {}
};
```

```
// === Match 3 (js/games/match3.js) ===

/**Ð
 * Match 3 ([•wómˆmˆNP)Ð
 * @author ch2077Ð
 * @version 1.2.0Ð
 * @date 2025-04-10 e°XžN†dropN „=R¨u;Ð
 * @date 2025-04-15 OîN†•Þs¯mˆ–dR$e-o h<v„•î˜˜Ð
 *Ð
 * 8x8•Qh<ÿ 6yÍ[•wóÿ N¤cbvø•»h<QÑ3•ÞÐ
 * mˆ–dT N e¹[•wóN „=ÿ zzOMNÎ˜v•èˆe–•g:e°[•wóÐ
 * u(BFShÀgå•Þ~¿ÿ j*zÖN$N*e¹T R R+bkÿ ÿ • _RhÀgå•Þ• Ð
 */Ð
const Match3Game = {Ð
  grid: [], size: 8,Ð
  gems: ['Ø=Ý4','Ø=Ý5','Ø=ßå','Ø=ßá','Ø=ßã','Ø=ßà'],  // emoji[•wóÿ ˜œ,rf \ e>gemColorsÐ
  gemColors: { 'Ø=Ý4':'#ff6b6b', 'Ø=Ý5':'#54a0ff', 'Ø=ßå':'#00d2a0', 'Ø=ßá':'#ffd93d', 'Ø=ßã
':'#a29bfe', 'Ø=ßà':'#ffa726' },Ð
  selected: null,Ð
  score: 0, moves: 20,Ð
  container: null, statsCb: null,Ð
  running: false,Ð
  swapping: false,Ð
Ð
  init(container, statsCb) {Ð
    this.container = container;Ð
    this.statsCb = statsCb;Ð
    this.score = 0;Ð
    this.moves = 20;Ð
    this.selected = null;Ð
    this.running = true;Ð
    this.swapping = false;Ð
    this.initGrid();Ð
    this.render();Ð
    this.updateStats();Ð
  },Ð
Ð
  initGrid() {Ð
    this.grid = Array.from({ length: this.size }, () =>Ð
      Array.from({ length: this.size }, () =>Ð
        this.gems[Math.floor(Math.random() * this.gems.length)]Ð
      )Ð
    );Ð
    // Remove initial matchesÐ
    let matches = this.findAllMatches();Ð
    while (matches.length > 0) {Ð
      for (const { r, c } of matches) {Ð
        this.grid[r][c] = this.gems[Math.floor(Math.random() * this.gems.length)];
Ð
      }Ð
      matches = this.findAllMatches();Ð
    }Ð
  },Ð
Ð
```

```
  render() {Ð
    let gridHTML = '';Ð
    for (let r = 0; r < this.size; r++) {Ð
      gridHTML += '<div class="match3-row" style="display:flex">';Ð
      for (let c = 0; c < this.size; c++) {Ð
        gridHTML += `<div class="match3-cell" id="m3_${r}_${c}" data-r="${r}" data
-c="${c}" style="Ð
          flex:1;aspect-ratio:1;display:flex;align-items:center;justify-content:ce
nter;Ð
          font-size:28px;cursor:pointer;border-radius:6px;Ð
          transition:transform 0.15s,opacity 0.3s;Ð
          ">${this.grid[r][c]}</div>`;Ð
      }Ð
      gridHTML += '</div>';Ð
    }Ð
Ð
    this.container.innerHTML = `Ð
      <div style="text-align:center;touch-action:manipulation">Ð
        <div style="color:var(--text2);font-size:13px;margin-bottom:8px">N¤cbvø•»[•wóÿ 3N*
NåN mˆ–d_—R </div>Ð
        <div class="match3-board" style="background:var(--surface2);padding:8px;bo
rder-radius:12px;max-width:360px;margin:0 auto">Ð
          ${gridHTML}Ð
        </div>Ð
        <div id="match3Msg" style="margin-top:10px;min-height:20px;font-size:14px;
font-weight:600;color:var(--accent2)"></div>Ð
      </div>`;Ð
Ð
    this.container.querySelectorAll('.match3-cell').forEach(cell => {Ð
      cell.addEventListener('click', () => {Ð
        if (!this.running || this.swapping) return;Ð
        const r = parseInt(cell.dataset.r);Ð
        const c = parseInt(cell.dataset.c);Ð
        this.selectCell(r, c);Ð
      });Ð
Ð
      // Touch drag supportÐ
      cell.addEventListener('touchstart', (e) => {Ð
        if (!this.running || this.swapping) return;Ð
        const r = parseInt(cell.dataset.r);Ð
        const c = parseInt(cell.dataset.c);Ð
        this._touchStart = { r, c, x: e.touches[0].clientX, y: e.touches[0].client
Y };Ð
      });Ð
      cell.addEventListener('touchend', (e) => {Ð
        if (!this.running || this.swapping || !this._touchStart) return;Ð
        const endX = e.changedTouches[0].clientX;Ð
        const endY = e.changedTouches[0].clientY;Ð
        const dx = endX - this._touchStart.x;Ð
        const dy = endY - this._touchStart.y;Ð
        const { r, c } = this._touchStart;Ð
        let tr = r, tc = c;Ð
        if (Math.abs(dx) > Math.abs(dy)) {Ð
          tc = dx > 0 ? c + 1 : c - 1;Ð
        } else {Ð
```

```
          tr = dy > 0 ? r + 1 : r - 1;Ð
        }Ð
        this.selectCell(r, c);Ð
        if (tr >= 0 && tr < this.size && tc >= 0 && tc < this.size) {Ð
          this.selectCell(tr, tc);Ð
        }Ð
        this._touchStart = null;Ð
      });Ð
    });Ð
  },Ð
Ð
  selectCell(r, c) {Ð
    // Clear previous selectionÐ
    this.container.querySelectorAll('.match3-cell').forEach(c =>Ð
      c.style.outline = 'none');Ð
Ð
    if (!this.selected) {Ð
      this.selected = { r, c };Ð
      const cell = document.getElementById(`m3_${r}_${c}`);Ð
      if (cell) cell.style.outline = '2px solid var(--accent)';Ð
      return;Ð
    }Ð
Ð
    const prev = this.selected;Ð
    this.selected = null;Ð
Ð
    // Check if adjacentÐ
    const dr = Math.abs(r - prev.r);Ð
    const dc = Math.abs(c - prev.c);Ð
    if ((dr === 1 && dc === 0) || (dr === 0 && dc === 1)) {Ð
      this.swapAndCheck(prev.r, prev.c, r, c);Ð
    } else {Ð
      this.selected = { r, c };Ð
      const cell = document.getElementById(`m3_${r}_${c}`);Ð
      if (cell) cell.style.outline = '2px solid var(--accent)';Ð
    }Ð
  },Ð
Ð
  async swapAndCheck(r1, c1, r2, c2) {Ð
    this.swapping = true;Ð
    this.swap(r1, c1, r2, c2);Ð
    await this.animateSwap(r1, c1, r2, c2);Ð
Ð
    const matches = this.findAllMatches();Ð
    if (matches.length === 0) {Ð
      // Swap backÐ
      this.swap(r1, c1, r2, c2);Ð
      await this.animateSwap(r1, c1, r2, c2);Ð
      document.getElementById('match3Msg').textContent = 'eàlÕmˆ–dÿ ';Ð
      setTimeout(() => { document.getElementById('match3Msg').textContent = ''; },
 1000);Ð
      this.swapping = false;Ð
      return;Ð
    }Ð
Ð
```

```
    document.getElementById('match3Msg').textContent = '';Ð
    this.moves--;Ð
Ð
    // Chain reactionsÐ
    while (matches.length > 0) {Ð
      for (const { r, c } of matches) {Ð
        this.score += 10;Ð
        const cell = document.getElementById(`m3_${r}_${c}`);Ð
        if (cell) { cell.style.opacity = '0'; cell.style.transform = 'scale(0.5)';
 }Ð
      }Ð
      await this.sleep(250);Ð
Ð
      // Remove and dropÐ
      const removed = new Set(matches.map(m => `${m.r},${m.c}`));Ð
      for (let c = 0; c < this.size; c++) {Ð
        let writeRow = this.size - 1;Ð
        for (let r = this.size - 1; r >= 0; r--) {Ð
          if (!removed.has(`${r},${c}`)) {Ð
            this.grid[writeRow][c] = this.grid[r][c];Ð
            writeRow--;Ð
          }Ð
        }Ð
        for (let r = writeRow; r >= 0; r--) {Ð
          this.grid[r][c] = this.gems[Math.floor(Math.random() * this.gems.length)
];Ð
        }Ð
      }Ð
      this.refreshDisplay();Ð
      await this.sleep(200);Ð
Ð
      const newMatches = this.findAllMatches();Ð
      // Only continue if we found new matchesÐ
      if (newMatches.length === matches.length) break; // prevent infinite loopÐ
      matches.length = 0;Ð
      for (const m of newMatches) matches.push(m);Ð
    }Ð
Ð
    this.refreshDisplay();Ð
    this.updateStats();Ð
    this.swapping = false;Ð
Ð
    if (this.moves <= 0) this.gameOver();Ð
  },Ð
Ð
  swap(r1, c1, r2, c2) {Ð
    const tmp = this.grid[r1][c1];Ð
    this.grid[r1][c1] = this.grid[r2][c2];Ð
    this.grid[r2][c2] = tmp;Ð
  },Ð
Ð
  animateSwap(r1, c1, r2, c2) {Ð
    const cell1 = document.getElementById(`m3_${r1}_${c1}`);Ð
    const cell2 = document.getElementById(`m3_${r2}_${c2}`);Ð
    if (cell1) {Ð
```

```
      cell1.style.transform = `translate(${(c2-c1)*100}%, ${(r2-r1)*100}%)`;Ð
      cell1.textContent = this.grid[r1][c1];Ð
    }Ð
    if (cell2) {Ð
      cell2.style.transform = `translate(${(c1-c2)*100}%, ${(r1-r2)*100}%)`;Ð
      cell2.textContent = this.grid[r2][c2];Ð
    }Ð
    return this.sleep(200);Ð
  },Ð
Ð
  refreshDisplay() {Ð
    for (let r = 0; r < this.size; r++) {Ð
      for (let c = 0; c < this.size; c++) {Ð
        const cell = document.getElementById(`m3_${r}_${c}`);Ð
        if (cell) {Ð
          cell.textContent = this.grid[r][c];Ð
          cell.style.opacity = '1';Ð
          cell.style.transform = 'scale(1)';Ð
        }Ð
      }Ð
    }Ð
  },Ð
Ð
  findAllMatches() {Ð
    const matched = new Set();Ð
    // HorizontalÐ
    for (let r = 0; r < this.size; r++) {Ð
      for (let c = 0; c < this.size - 2; c++) {Ð
        if (this.grid[r][c] === this.grid[r][c+1] && this.grid[r][c] === this.grid
[r][c+2]) {Ð
          let end = c + 2;Ð
          while (end + 1 < this.size && this.grid[r][end + 1] === this.grid[r][c])
 end++;Ð
          for (let i = c; i <= end; i++) matched.add(`${r},${i}`);Ð
        }Ð
      }Ð
    }Ð
    // VerticalÐ
    for (let c = 0; c < this.size; c++) {Ð
      for (let r = 0; r < this.size - 2; r++) {Ð
        if (this.grid[r][c] === this.grid[r+1][c] && this.grid[r][c] === this.grid
[r+2][c]) {Ð
          let end = r + 2;Ð
          while (end + 1 < this.size && this.grid[end + 1][c] === this.grid[r][c])
 end++;Ð
          for (let i = r; i <= end; i++) matched.add(`${i},${c}`);Ð
        }Ð
      }Ð
    }Ð
    return Array.from(matched).map(s => {Ð
      const [r, c] = s.split(',').map(Number);Ð
      return { r, c };Ð
    });Ð
  },Ð
Ð
```

```
  sleep(ms) { return new Promise(r => setTimeout(r, ms)); },Đ
Đ
  updateStats() {Đ
    if (this.statsCb) this.statsCb({ '_—R ': this.score, 'RiOYkeep': this.moves });Đ
  },Đ
Đ
  gameOver() {Đ
    this.running = false;Đ
    const div = document.createElement('div');Đ
    div.className = 'game-over-overlay';Đ
    div.innerHTML = `<h3>n8b ~Óg_</h3><p>_—R : ${this.score}</p><button class="btn" id="
replayBtn">Q•geN k!</button>`;Đ
    this.container.style.position = 'relative';Đ
    this.container.appendChild(div);Đ
    document.getElementById('replayBtn').addEventListener('click', () => {Đ
      this.init(this.container, this.statsCb);Đ
    });Đ
  },Đ
Đ
  destroy() { this.running = false; }Đ
};Đ
```